

Guía previa del laboratorio 3: Sincronización con múltiples consumidores y planificación de procesos

Curso: 1INF29 — Sistemas Operativos · PUCP · 2026-1

Tema: Hilos POSIX, semáforos no nombrados y algoritmos de planificación de CPU

Referencia del syllabus: Capítulo 4 — Procesos y comunicación entre procesos (secciones 4.5 y 4.6); Capítulo 5 — Planificación y despacho de procesos

Bibliografía: Tanenbaum, *Modern Operating Systems* (2015), pp. 97–110, 149–172, 435–465

Coordinador del laboratorio: Alberto M. Torreblanca Villavicencio

1. Propósito de esta guía

Esta guía debe ser estudiada **antes** de asistir al laboratorio. Su objetivo es que llegues con los conceptos resueltos y las herramientas identificadas, de modo que en la sesión presencial puedas concentrarte en implementar y depurar.

El laboratorio 3 combina dos áreas que el curso ha tratado por separado: la **sincronización entre hilos con semáforos** (Cap. 4) y la **planificación de procesos** (Cap. 5). La guía cubre ambas en bloques independientes y termina con una autoevaluación que debes poder responder antes de la sesión.

2. Temas que se trabajarán en el laboratorio

El laboratorio cubre dos áreas técnicas que debes dominar antes de la sesión:

Concurrencia con hilos POSIX y semáforos no nombrados. Trabajarás con varios hilos del mismo proceso que se coordinan a través de un recurso compartido de capacidad limitada. Necesitarás manejar el patrón productor-consumidor con múltiples consumidores intercambiables, proteger secciones críticas con semáforos en modo mutex, y decidir cuándo conviene una espera bloqueante y cuándo una espera no bloqueante. Las primitivas que debes conocer son `pthread_create()`, `pthread_join()`, `sem_init()`, `sem_wait()`, `sem_post()`, `sem_trywait()` y `sem_destroy()`.

Algoritmos de planificación de CPU. Trabajarás con simulaciones deterministas en C de dos algoritmos de scheduling no apropiativos: FCFS y SJF. Necesitarás calcular a mano el turnaround, el tiempo de espera y el diagrama de Gantt para un conjunto de cuatro procesos, e implementar la lógica equivalente en código. Los conceptos centrales son llegada, ráfaga, tiempo de espera de la CPU y métricas promedio.

3. Lo que debes revisar antes del laboratorio

3.1. Conceptos teóricos

Repasa los siguientes temas de Tanenbaum (2015):

- **Sección 2.4 — Planificación de procesos** (pp. 149–172). Algoritmos FCFS y SJF (no apropiativos). Métricas: turnaround y tiempo de espera.
- **Sección 2.2.3 — Hilos POSIX (pthreads)** (pp. 105–110). Diferencia entre `fork` y

- pthread_create. Modelo de memoria compartida entre hilos del mismo proceso.
- **Sección 2.3.5 — Semáforos** (pp. 130–132). Operaciones down (sem_wait) y up (sem_post). Semáforos contadores vs. semáforos binarios (mutex).
- **Sección 2.5.5 — Problemas de IPC** (pp. 167–170). Patrón productor-consumidor con capacidad limitada y consumidor que duerme cuando no hay trabajo.

3.2. Páginas de manual

Dedica entre 10 y 15 minutos a leer estas páginas. No es necesario memorizar las firmas, sí identificar parámetros y valores de retorno:

```
man pthread_create # Crear un hilo POSIX
man pthread_join  # Esperar a que un hilo termine
man sem_init       # Inicializar un semáforo SIN nombre (entre hilos)
man sem_wait       # Decrementar (bloqueante)
man sem_trywait    # Decrementar (NO bloqueante) — útil para patrones de rechazo
man sem_post       # Incrementar (puede despertar a un hilo bloqueado)
man sem_destroy    # Liberar recursos del semáforo
bash
```

***Nota importante.** En el laboratorio anterior usaste sem_open (semáforos con nombre, visibles entre procesos distintos). En esta sesión trabajarás con sem_init (semáforos sin nombre, compartidos entre **hilos** del mismo proceso). Asegúrate de entender esta distinción antes de avanzar.*

3.3. Herramientas de verificación

Familiarízate con estos comandos antes del laboratorio:

```
ps -elf | grep nombre_programa # Lista los HILOS de cada proceso (columna LWP)
top -H -p <PID>                # Ver hilos en tiempo real de un proceso
gcc -Wall -Wextra prog.c -lpthread # Compilar habilitando todos los warnings
```

Para los ejercicios de planificación no necesitas herramientas especiales del sistema: la simulación es puramente determinista en C estándar.

4. Ejemplo 1: hilos POSIX básicos

Este programa crea tres hilos. Cada hilo recibe un ID, imprime un mensaje y termina. Es el equivalente con hilos del primer programa con fork() que escribiste en cursos anteriores.

```
/* ejemplo1_pthreads.c
 * Crea 3 hilos. Cada hilo recibe un ID y trabaja independientemente.
 * Compilar: gcc ejemplo1_pthreads.c -o ejemplo1 -lpthread
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_HILOS 3

/* La función de un hilo SIEMPRE recibe un void* y retorna void*.
 * El argumento llega como puntero, así que hay que castearlo. */
void *trabajador(void *arg) {
    int id = *((int *)arg);
    printf("Hilo %d: comencé a trabajar.\n", id);
    sleep(1); /* Simula trabajo. */
    printf("Hilo %d: terminé.\n", id);
    return NULL;
}
```

```

int main() {
    pthread_t hilos[NUM_HILOS];
    int ids[NUM_HILOS];

    for (int i = 0; i < NUM_HILOS; i++) {
        ids[i] = i;
        /* pthread_create:
         * - &hilos[i]: dónde guardar el handle del nuevo hilo
         * - NULL: atributos por defecto
         * - trabajador: función que ejecutará el hilo
         * - &ids[i]: argumento que se le pasa al hilo */
        if (pthread_create(&hilos[i], NULL, trabajador, &ids[i]) != 0) {
            perror("pthread_create");
            exit(EXIT_FAILURE);
        }
    }

    /* pthread_join bloquea hasta que el hilo termine.
     * Es el equivalente a wait() en procesos. */
    for (int i = 0; i < NUM_HILOS; i++) {
        pthread_join(hilos[i], NULL);
    }

    printf("Main: todos los hilos terminaron.\n");
    return 0;
}

```

Possible salida (el orden entre hilos puede variar):

```

Hilo 0: comencé a trabajar.
Hilo 1: comencé a trabajar.
Hilo 2: comencé a trabajar.
Hilo 0: terminé.
Hilo 1: terminé.
Hilo 2: terminé.
Main: todos los hilos terminaron.

```

4.1. Preguntas sobre el ejemplo 1

P1.1. ¿Por qué cada hilo recibe `&ids[i]` y no simplemente `&i`? Si pasas `&i`, ¿qué problema podría aparecer? Pruébalo y observa qué imprime cada hilo.

P1.2. Con `fork()`, los procesos hijos no comparten variables con el padre (cada uno tiene su copia). Con `pthread_create()`, los hilos comparten varios recursos del proceso entre ellos, las variables globales. Ejecuta `ps -eLf` mientras el programa duerme y observa la columna `LWP`. ¿Cuántos `LWP` aparecen para tu programa?

P1.3. ¿Qué sucede si eliminas el `pthread_join()`? ¿Pueden los hilos quedar sin terminar? Pruébalo retirando un `sleep` largo de `main`.

5. Ejemplo 2: sincronización entre hilos con `sem_init`

Este programa usa un semáforo **sin nombre** (`sem_init`) para que un hilo señalice a otro. Compáralo mentalmente con el equivalente que viste en la guía anterior, donde `sem_open` cumplía un rol similar pero entre procesos distintos.

```

/* ejemplo2_sem_init.c
 * Sincronización entre dos hilos con un semáforo sin nombre.
 * Compilar: gcc ejemplo2_sem_init.c -o ejemplo2 -lpthread
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

/* Variable global compartida por los dos hilos. */
sem_t listo;
int dato = 0;

void *productor(void *arg) {
    (void)arg;
    printf("Productor: preparando dato...\n");
    sleep(2); /* Simula trabajo costoso. */
    dato = 42;
    printf("Productor: dato = %d, notificando.\n", dato);
    sem_post(&listo); /* Despierta al consumidor. */
    return NULL;
}

void *consumidor(void *arg) {
    (void)arg;
    printf("Consumidor: esperando dato...\n");
    sem_wait(&listo); /* Bloquea hasta el sem_post. */
    printf("Consumidor: dato recibido = %d\n", dato);
    return NULL;
}

int main() {
    /* sem_init(sem, pshared, valor_inicial)
     * - pshared = 0: el semáforo solo se usa entre HILOS del proceso.
     * - pshared = 1: también entre procesos (requiere memoria compartida).
     * - valor_inicial = 0: el primer sem_wait bloqueará. */
    if (sem_init(&listo, 0, 0) != 0) {
        perror("sem_init");
        exit(EXIT_FAILURE);
    }

    pthread_t h_prod, h_cons;
    pthread_create(&h_prod, NULL, productor, NULL);
    pthread_create(&h_cons, NULL, consumidor, NULL);

    pthread_join(h_prod, NULL);
    pthread_join(h_cons, NULL);

    sem_destroy(&listo);
    return 0;
}

```

Salida esperada (siempre en este orden gracias al semáforo):

```

Consumidor: esperando dato...
Productor: preparando dato...
Productor: dato = 42, notificando.
Consumidor: dato recibido = 42

```

5.1. Preguntas sobre el ejemplo 2

P2.1. ¿Qué pasaría si inicializaras el semáforo en 1 en vez de 0? Razona sobre el comportamiento del consumidor en ese caso.

P2.2. Compara `sem_init` con `sem_open`. ¿Cuándo usarías cada uno? Llena esta tabla mental:

Situación	¿Qué primitiva uso?
Dos hilos del mismo proceso se sincronizan	—
Padre e hijo creados con <code>fork()</code> se sincronizan	—
Dos programas independientes se coordinan por nombre	—

P2.3. ¿Por qué `sem_destroy` solo se llama una vez en `main` y no en cada hilo? ¿Qué pasaría si lo llamaras dentro de un hilo antes de que el otro haya terminado?

6. Ejemplo 3: `sem_trywait` y el patrón de rechazo

Este ejemplo demuestra `sem_trywait`, la operación **no bloqueante** sobre un semáforo. A diferencia de `sem_wait`, si el recurso no está disponible no bloquea al hilo: retorna de inmediato con un error. Es la primitiva apropiada cuando un sistema con capacidad limitada debe **rechazar** nuevas peticiones en lugar de hacerlas esperar.

```
/* ejemplo3_trywait.c
 * Demuestra la diferencia entre sem_wait y sem_trywait.
 * Modela una sala con 3 asientos: si está llena, los clientes se van.
 * Compilar: gcc ejemplo3_trywait.c -o ejemplo3 -lpthread
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <errno.h>

#define ASIENTOS 3
#define NUM_CLIENTES 6

sem_t sala;

void *cliente(void *arg) {
    int id = *((int *)arg);

    /* sem_trywait intenta decrementar el semáforo.
     * Si vale 0, retorna -1 inmediatamente con errno = EAGAIN.
     * NO bloquea al hilo. */
    if (sem_trywait(&sala) == -1) {
        if (errno == EAGAIN) {
            printf("Cliente %d: sala llena, me retiro.\n", id);
            return NULL;
        }
        perror("sem_trywait");
        return NULL;
    }

    /* Si llegamos aquí, conseguimos un asiento. */
    printf("Cliente %d: tomé asiento.\n", id);
    sleep(2); /* Tiempo en la sala. */
    printf("Cliente %d: libero el asiento.\n", id);
    sem_post(&sala);
    return NULL;
}

int main() {
    sem_init(&sala, 0, ASIENTOS);

    pthread_t hilos[NUM_CLIENTES];
    int ids[NUM_CLIENTES];
```

```

/* Lanzamos los 6 clientes casi simultáneamente.
 * Como hay solo 3 asientos, 3 deberían retirarse. */
for (int i = 0; i < NUM_CLIENTES; i++) {
    ids[i] = i;
    pthread_create(&hilos[i], NULL, cliente, &ids[i]);
}

for (int i = 0; i < NUM_CLIENTES; i++) {
    pthread_join(hilos[i], NULL);
}

sem_destroy(&sala);
return 0;
}

```

Possible salida (los rechazados pueden ser otros, pero deben ser tres):

```

Cliente 0: tomé asiento.
Cliente 1: tomé asiento.
Cliente 2: tomé asiento.
Cliente 3: sala llena, me retiro.
Cliente 4: sala llena, me retiro.
Cliente 5: sala llena, me retiro.
Cliente 0: libero el asiento.
Cliente 1: libero el asiento.
Cliente 2: libero el asiento.

```

6.1. Preguntas sobre el ejemplo 3

P3.1. Cambia `sem_trywait` por `sem_wait` y vuelve a ejecutar. ¿Cuántos clientes son rechazados ahora? ¿Por qué? ¿Cuál es el costo de este cambio en un servidor real con miles de clientes?

P3.2. ¿Qué clientes específicos terminan rechazados en cada ejecución? Ejecuta el programa cinco veces y registra el resultado. ¿Por qué no es siempre el mismo conjunto?

P3.3. Existe también `sem_timedwait`, que espera durante un tiempo máximo y luego falla. ¿En qué escenario sería preferible a `sem_trywait`? Da un ejemplo concreto del mundo real.

7. Ejemplo 4: cálculo manual de planificación FCFS y SJF

El cálculo manual es la mejor herramienta para entender un algoritmo de planificación antes de codificarlo. Si tu programa no reproduce los números que obtuviste a mano, sabrás que hay un bug; si los reproduce, tendrás confianza en tu implementación. Esta sección desarrolla el mismo conjunto de procesos que usarás en el Problema 2 del laboratorio.

7.1. Conjunto de procesos

PID	Llegada	Ráfaga
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Observa que P3 tiene una ráfaga muy corta (1) pero llega tarde ($t=4$). Este proceso es la clave para entender la diferencia entre FCFS y SJF: FCFS lo castigará porque ya hay procesos más largos ejecutándose, mientras que SJF lo atenderá pronto por ser el más corto entre los disponibles.

7.2. Cálculo de FCFS (First Come, First Served)

FCFS atiende en orden de llegada, sin interrupciones. Construyamos el diagrama de Gantt paso a paso:

t=0: solo P1 ha llegado → ejecuta P1 hasta su ráfaga (7)
t=7: P2, P3 y P4 ya llegaron, pero FCFS respeta la cola → ejecuta P2 (4)
t=11: ejecuta P3 (1)
t=12: ejecuta P4 (4)
t=16: fin

Diagrama de Gantt:

|P1|P1|P1|P1|P1|P1|P1|P2|P2|P2|P2|P3|P4|P4|P4|P4|
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

PID	Llegada	Fin	Turnaround	Espera
P1	0	7	$7 - 0 = 7$	$7 - 7 = 0$
P2	2	11	$11 - 2 = 9$	$9 - 4 = 5$
P3	4	12	$12 - 4 = 8$	$8 - 1 = 7$
P4	5	16	$16 - 5 = 11$	$11 - 4 = 7$

Promedios: turnaround = $(7+9+8+11)/4 = 8.75$; espera = $(0+5+7+7)/4 = 4.75$.

Nota como P3, que solo necesita 1 unidad de CPU, termina esperando 7 unidades en la cola. Es el efecto convoy: un proceso largo (P1 con ráfaga 7) bloquea a todos los procesos que llegan después, incluso si son muy cortos.

7.3. Cálculo de SJF (no apropiativo)

SJF, en cada decisión, elige al proceso ya llegado con la **ráfaga más corta**. Una decisión solo se toma cuando termina el proceso actual.

t=0: solo P1 ha llegado → ejecuta P1 (no hay opción)
t=7: ya llegaron P2 (ráfaga 4), P3 (ráfaga 1) y P4 (ráfaga 4)
SJF elige P3 por tener la ráfaga más corta
t=8: quedan P2 (ráfaga 4) y P4 (ráfaga 4) → empate, elige menor PID → P2
t=12: solo queda P4 → ejecuta P4
t=16: fin

Diagrama de Gantt:

|P1|P1|P1|P1|P1|P1|P1|P3|P2|P2|P2|P2|P4|P4|P4|P4|
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

PID	Llegada	Fin	Turnaround	Espera
P1	0	7	$7 - 0 = 7$	$7 - 7 = 0$
P2	2	12	$12 - 2 = 10$	$10 - 4 = 6$
P3	4	8	$8 - 4 = 4$	$4 - 1 = 3$
P4	5	16	$16 - 5 = 11$	$11 - 4 = 7$

Promedios: turnaround = $(7+10+4+11)/4 = 8.00$; espera = $(0+6+3+7)/4 = 4.00$.

Observa la diferencia con FCFS: P3 ahora solo espera 3 unidades (contra 7 en FCFS) y su turnaround baja de 8 a 4. SJF mejora las métricas promedio porque atiende primero los procesos cortos, reduciendo el tiempo que otros esperan. Este es el caso clásico donde SJF supera claramente a FCFS.

7.4. Comparación

Algoritmo	Turnaround promedio	Espera promedio
FCFS	8.75	4.75
SJF	8.00	4.00

SJF mejora ambas métricas. Esto es consistente con la propiedad teórica de que SJF es **óptimo en turnaround promedio** entre los algoritmos no apropiativos. Sin embargo, fíjate que P4 no mejora: en ambos algoritmos termina en $t=16$ y su espera sigue siendo 7. SJF beneficia a los procesos cortos a costa de los largos.

7.5. Preguntas sobre el ejemplo 4

P4.1. Verifica los cálculos anteriores con papel y lápiz. Si encuentras un error, corrígelo y verifica tus números con los valores esperados del laboratorio.

P4.2. Si quisieras adaptar el simulador FCFS para implementar SJF, ¿qué función reemplazarías y qué cambios introducirías? ¿La estructura `Proceso` sería suficiente o necesitarías más campos?

P4.3. SJF mejora las métricas promedio pero P4 no se beneficia en absoluto. ¿Por qué? ¿Qué proceso sí se beneficia y en qué magnitud?

8. Esqueleto de simulador en C: práctica adicional

Como ejercicio de práctica, este es un simulador FCFS funcional que reproduce el cálculo de la sección 7.2. Estudia su estructura, compílalo y verifica que su salida coincida con tus cálculos manuales. La estructura `Proceso` y la separación entre la lógica del algoritmo y la impresión te servirán como modelo para diseñar simulaciones de cualquier algoritmo de planificación.

```
/* ejemplo5_fcfs.c
 * Simulador FCFS básico. Sirve como punto de partida para SJF.
 * Compilar: gcc ejemplo5_fcfs.c -o ejemplo5
 */
#include <stdio.h>

typedef struct {
    int pid;
    int llegada;
    int rafaga;
    int inicio;
    int fin;
    int turnaround;
    int espera;
} Proceso;

void fcfs(Proceso p[], int n) {
    int tiempo = 0;
    for (int i = 0; i < n; i++) {
        if (tiempo < p[i].llegada) {
            tiempo = p[i].llegada; /* CPU ociosa. */
        }
        p[i].inicio = tiempo;
        p[i].fin = tiempo + p[i].rafaga;
        p[i].turnaround = p[i].fin - p[i].llegada;
        p[i].espera = p[i].turnaround - p[i].rafaga;
        tiempo = p[i].fin;
    }
}
```



```

    }
}

void imprimir(const char *nombre, Proceso p[], int n) {
    printf("=== %s ===\n", nombre);
    printf("PID  Llegada  Rafaga  Inicio  Fin    Turnaround  Espera\n");
    int sum_t = 0, sum_e = 0;
    for (int i = 0; i < n; i++) {
        printf(" %d      %2d      %2d      %2d      %2d      %2d\n",
            p[i].pid, p[i].llegada, p[i].rafaga,
            p[i].inicio, p[i].fin, p[i].turnaround, p[i].espera);
        sum_t += p[i].turnaround;
        sum_e += p[i].espera;
    }
    printf("Promedio turnaround: %.2f\n", (double)sum_t / n);
    printf("Promedio espera:      %.2f\n", (double)sum_e / n);
}

int main() {
    Proceso p[] = {
        {1, 0, 7, 0, 0, 0, 0},
        {2, 2, 4, 0, 0, 0, 0},
        {3, 4, 1, 0, 0, 0, 0},
        {4, 5, 4, 0, 0, 0, 0}
    };
    int n = sizeof(p) / sizeof(p[0]);

    fcfs(p, n);
    imprimir("FCFS", p, n);
    return 0;
}

```

8.1. Preguntas sobre el esqueleto

P5.1. Compila y ejecuta el programa. ¿La salida coincide con tu cálculo manual de la sección 7.2? Si no coincide, ¿dónde está el error: en tu cálculo o en el código?

P5.2. Si simularas FCFS y SJF sobre el mismo arreglo `p[]` en una sola ejecución, ¿qué problema aparecería con los campos `inicio`, `fin`, `turnaround` y `espera`? ¿Cómo lo resolverías?

9. Resumen de primitivas que usarás en el laboratorio

Función	Cabecera	Propósito	Flag para enlazar
<code>pthread_create()</code>	<code><pthread.h></code>	Crear un hilo	<code>-lpthread</code>
<code>pthread_join()</code>	<code><pthread.h></code>	Esperar a que un hilo termine	<code>-lpthread</code>
<code>sem_init()</code>	<code><semaphore.h></code>	Inicializar semáforo sin nombre	<code>-lpthread</code>
<code>sem_wait()</code>	<code><semaphore.h></code>	Decrementar (bloqueante)	<code>-lpthread</code>
<code>sem_trywait()</code>	<code><semaphore.h></code>	Decrementar (no bloqueante)	<code>-lpthread</code>
<code>sem_post()</code>	<code><semaphore.h></code>	Incrementar	<code>-lpthread</code>
<code>sem_destroy()</code>	<code><semaphore.h></code>	Liberar el semáforo	<code>-lpthread</code>

Para los ejercicios de planificación no necesitas primitivas del sistema: solo C estándar con `<stdio.h>`.

10. Autoevaluación: ¿estás listo para el laboratorio?

Antes de asistir, debes poder responder estas preguntas **sin consultar el material**:

1. ¿Cuál es la diferencia entre `sem_init` y `sem_open`? ¿En qué escenario usarías cada uno?
2. ¿Cuál es la diferencia entre `sem_wait` y `sem_trywait`? ¿Cuándo es preferible cada uno?
3. ¿Por qué dos hilos del mismo proceso **no necesitan** memoria compartida POSIX para compartir variables?
4. ¿Cuándo FCFS y SJF dan el mismo resultado? ¿Cuándo difieren?
5. Dado el conjunto de cuatro procesos de la sección 7.1, ¿puedes calcular a mano el turnaround promedio de FCFS y SJF sin equivocarte?
6. ¿Por qué SJF se considera "óptimo en turnaround promedio" pero rara vez se implementa en sistemas reales?

Si puedes responder al menos cinco de las seis preguntas con confianza, estás preparado para el laboratorio.

11. Preparación del entorno

Antes de la sesión, verifica que tu entorno funcione ejecutando:

```
# Verificar GCC
gcc --version

# Compilar el ejemplo 1 como prueba
gcc ejemplo1_pthreads.c -o ejemplo1 -lpthread
./ejemplo1

# Verificar que ves los hilos en ps
./ejemplo1 &
ps -eLf | grep ejemplo1
wait
```

Si `gcc` no está instalado, ejecuta:

```
sudo apt install build-essential
```

Si `pthread.h` no se encuentra, instala los headers:

```
sudo apt install libc6-dev
```